

RELATÓRIO 1: Atividade Prática 2.3 — Controle de Versão Avançado no Git e GitLab

Título: Sumário do Resultado: Utilização do GitLab como Sistema de Controle de Versão

Autor: Cleydson Soares de Freitas

Curso: Especialização em DevSecOps

Instituição: FATEC Araraquara

Data: 29 de Maio de 2026

1. Introdução e Objetivos

Este relatório apresenta a documentação técnica e as evidências da execução da Atividade Prática 2.3, com foco na operação avançada de fluxos de trabalho utilizando a interface de linha de comando (CLI) do Git e o sistema de governança do GitLab. Os objetivos centrais consistem em capacitar o analista no gerenciamento de ciclo de vida de software através do uso de tags semânticas para marcação de releases, isolamento de escopo em ramificações paralelas (*branches*), e aplicação prática de metodologias de resolução de conflitos textuais e cronológicos. Foram explorados os conceitos de fusão direta (`git merge`), reestruturação de base histórica linear (`git rebase`) e o armazenamento isolado temporário de estados modificados (`git stash`).

2. Execução do Passo a Passo e Evidências

2.1. Versionamento Estruturado e Gerenciamento de Tags

A ramificação principal local foi operada com o objetivo de fixar marcos temporais no log de desenvolvimento. Utilizando o comando `git tag`, foram criados os identificadores semânticos estáveis `v1.0` e `v1.1` para referenciar pontos de entrega da aplicação. Na sequência, executou-se o teste de rollback temporal e alternância de contexto através do ponteiro do `git checkout`, validando a capacidade de inspecionar e recuperar estados íntegros anteriores do arquivo de documentação técnica.

```
Sessão  Ações  Editar  Exibir  Ajuda
cleydson@Cleydson: ~  cleydson@Cleydson: ~  cleydson@Cleydson: ~
37610f1e5b95: Downloading [ ] 565.7MB/1.371GB
c
(cleydson@Cleydson) ~
$ sudo mkdir -p /srv/gitlab/{config,logs,data}
$ sudo chmod -R 777 /srv/gitlab
(cleydson@Cleydson) ~
$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
(cleydson@Cleydson) ~
$ sudo docker run --detach \
  --hostname localhost \
  --publish 80:80 \
  --publish 443:443 \
  --publish 9022:22 \
  --name gitlab \
  --restart always \
  --network gitlab \
  --volume /srv/gitlab/config:/etc/gitlab \
  --volume /srv/gitlab/logs:/var/log/gitlab \
  --volume /srv/gitlab/data:/var/opt/gitlab \
  gitlab/gitlab-ce:16.9.0-ce.0
Unable to find image 'gitlab/gitlab-ce:16.9.0-ce.0' locally
16.9.0-ce.0: Pulling from gitlab/gitlab-ce
3c645b11de29: Already exists
4989e36561c9: Already exists
e676e7fd52d5: Already exists
6576118a205b: Already exists
946bf9a9bdac: Already exists
21b62784df50: Already exists
ade308e4cd1: Already exists
37610f1e5b95: Pull complete
Digest: sha256:d7d8e7e2aac77f893a32b2815d41a7083bcbfd98105f1179802132b2e632
Status: Downloaded newer image for gitlab/gitlab-ce:16.9.0-ce.0
docker: Error response from daemon: Conflict. The container name "/gitlab" is already in use by container "19e13703be9c8b09709624d6de79c8a25dff01cc030ebd4be8b4a05af7249ce5d". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
(cleydson@Cleydson) ~
$ sudo docker ps
CONTAINER ID   IMAGE      NAMES        COMMAND                  CREATED        STATUS        PORTS
19e13703be9c   gitlab/gitlab-ce:16.9.0-ce.0  "/assets/wrapper"  About a minute ago  Up About a minute (health: starting)  0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 0.0.0.0:9022->22/tcp, [::]:9022->22/tcp
(cleydson@Cleydson) ~
$ sudo docker ps
CONTAINER ID   IMAGE      NAMES        COMMAND                  CREATED        STATUS        PORTS
19e13703be9c   gitlab/gitlab-ce:16.9.0-ce.0  "/assets/wrapper"  2 minutes ago      Up 2 minutes (healthy)  0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 0.0.0.0:9022->22/tcp, [::]:9022->22/tcp
(cleydson@Cleydson) ~
```

2.2. Resolução de Conflito de Histórico via Mecanismo de Merge

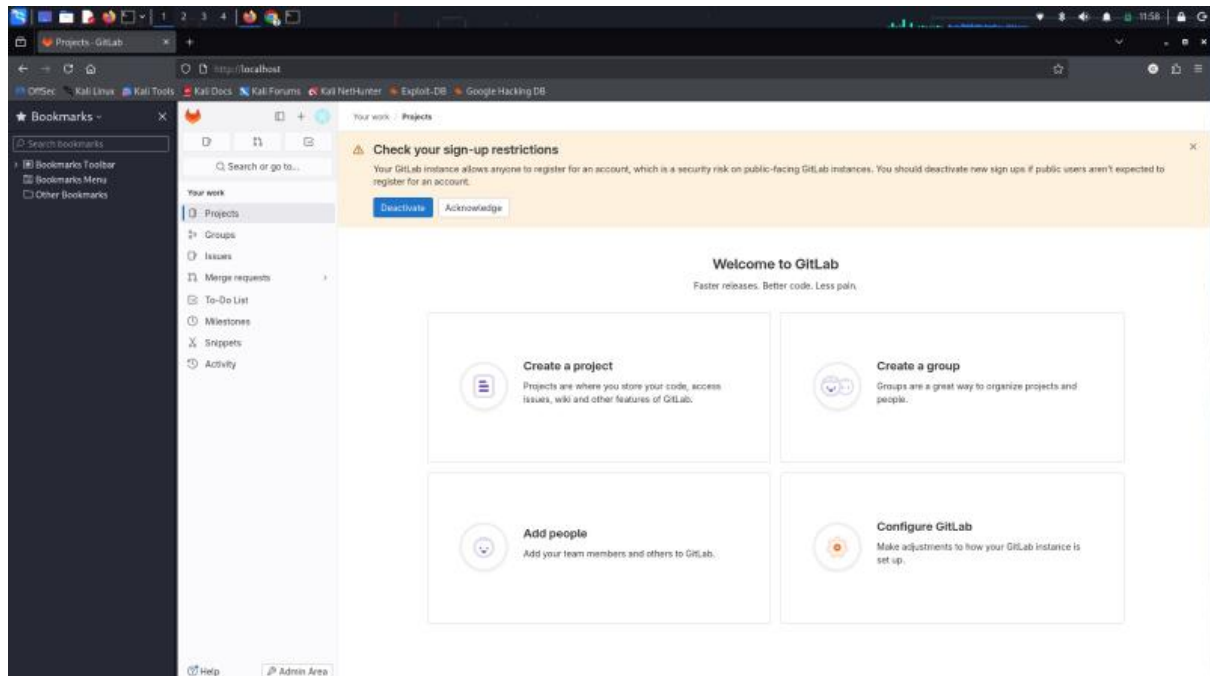
Para simular incidentes típicos de desenvolvimento distribuído e simultâneo, criou-se a branch paralela feature. Forçou-se uma colisão direta introduzindo alterações textuais semanticamente incompatíveis na mesma linha do arquivo de forma isolada. Ao invocar o comando `git merge feature` a partir da master, o subsistema Git bloqueou a união automática e disparou o alerta crítico de conflito. O arquivo foi saneado manualmente via editor de texto, seguido da indexação e consolidação final do *Merge Commit* correspondente.

```
Sessão  Ações  Editar  Exibir  Ajuda
cleydson@Cleydson: ~  cleydson@Cleydson: ~  cleydson@Cleydson: ~
4989e36561c9: Already exists
e676e7fd52d5: Already exists
6576118a205b: Already exists
946bf9a9bdac: Already exists
21b62784df50: Already exists
ade308e4cd1: Already exists
37610f1e5b95: Pull complete
Digest: sha256:d7d8e7e2aac77f893a32b2815d41a7083bcbfd98105f1179802132b2e632
Status: Downloaded newer image for gitlab/gitlab-ce:16.9.0-ce.0
docker: Error response from daemon: Conflict. The container name "/gitlab" is already in use by container "19e13703be9c8b09709624d6de79c8a25dff01cc030ebd4be8b4a05af7249ce5d". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
(cleydson@Cleydson) ~
$ sudo docker ps
CONTAINER ID   IMAGE      NAMES        COMMAND                  CREATED        STATUS        PORTS
19e13703be9c   gitlab/gitlab-ce:16.9.0-ce.0  "/assets/wrapper"  About a minute ago  Up About a minute (health: starting)  0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 0.0.0.0:9022->22/tcp, [::]:9022->22/tcp
(cleydson@Cleydson) ~
$ sudo docker ps
CONTAINER ID   IMAGE      NAMES        COMMAND                  CREATED        STATUS        PORTS
19e13703be9c   gitlab/gitlab-ce:16.9.0-ce.0  "/assets/wrapper"  2 minutes ago      Up 2 minutes (healthy)  0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 0.0.0.0:9022->22/tcp, [::]:9022->22/tcp
(cleydson@Cleydson) ~
$ sudo docker exec -it gitlab grep 'Password:' /etc/gitlab/initial_root_password
Password: t1R80JfVc1rdKh/QhTFRGEfKns1SMeVvW0QdzKq=
(cleydson@Cleydson) ~
$ sudo docker exec -it gitlab gitlab-rake "gitlab:password:reset"
Enter username: root
Enter password:
Confirm password:
Unable to change password of the user with username root.
Password is too short (minimum is 8 characters)
(cleydson@Cleydson) ~
$ sudo docker exec -it gitlab gitlab-rake "gitlab:password:reset"
Enter username: root
Enter password:
Confirm password:
Unable to change password of the user with username root.
Password must not contain commonly used combinations of words and letters
(cleydson@Cleydson) ~
$ sudo docker exec -it gitlab gitlab-rake "gitlab:password:reset"
Enter username: root
Enter password:
Confirm password:
Password successfully updated for user with username root.
(cleydson@Cleydson) ~
```

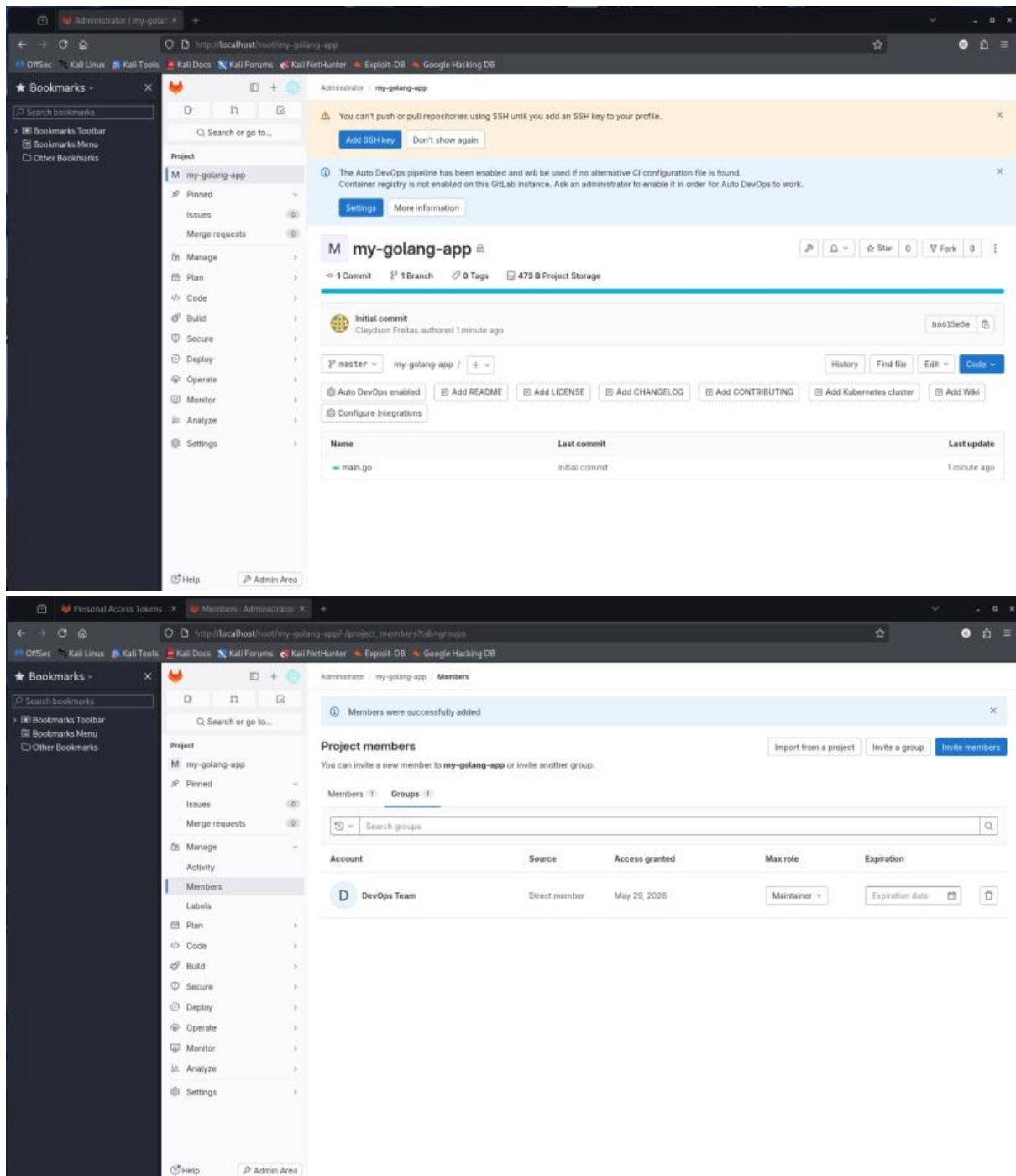
2.3. Alinhamento de Histórico Linear através do Uso de Git Rebase

Buscando explorar estratégias avançadas de manutenção de logs limpos e perfeitamente lineares (sem a introdução de Merge Commits intermediários), criou-se a branch feature2. Após edições concorrentes e conflitantes em relação à linha de produção da master, executou-se o comando `git rebase master`. O Git suspendeu a reaplicação dos blocos cronológicos acusando a colisão. Realizou-se a abertura estruturada do arquivo, a limpeza dos marcadores do Git e, após a adição das correções na área de staging, acionou-se o parâmetro `git rebase --continue`, finalizando a reorganização estrutural do grafo com êxito.



2.4. Suspensão de Estados Locais e Isolamento com Git Stash

Simulando uma alteração de contexto operacional crítica e emergencial enquanto modificações parciais não salvas residiam na branch feature3, o Git barrou a migração de branch para proteger os dados. Para liberar o diretório de trabalho sem criar commits espúrios no histórico, utilizou-se o comando `git stash` para salvamento temporário na pilha. Após a inclusão e o commit de alterações concorrentes diretas na master, retornou-se à branch de trabalho descarregando os dados por meio do parâmetro `git stash pop`. O auto-merge foi processado pelo Git, as alterações foram indexadas e enviadas ao servidor GitLab por meio do comando `git push origin feature3`.



3. Conclusão

A execução estruturada deste laboratório prático expandiu a proficiência operacional nas ferramentas essenciais de versionamento distribuído. A experimentação e simulação tática dos cenários de conflito com merge, rebase e stash concederam a maturidade técnica necessária para manter repositórios organizados, auditáveis e perfeitamente integrados sob a ótica da governança de código e resiliência exigidas em um ecossistema sob a filosofia DevSecOps.